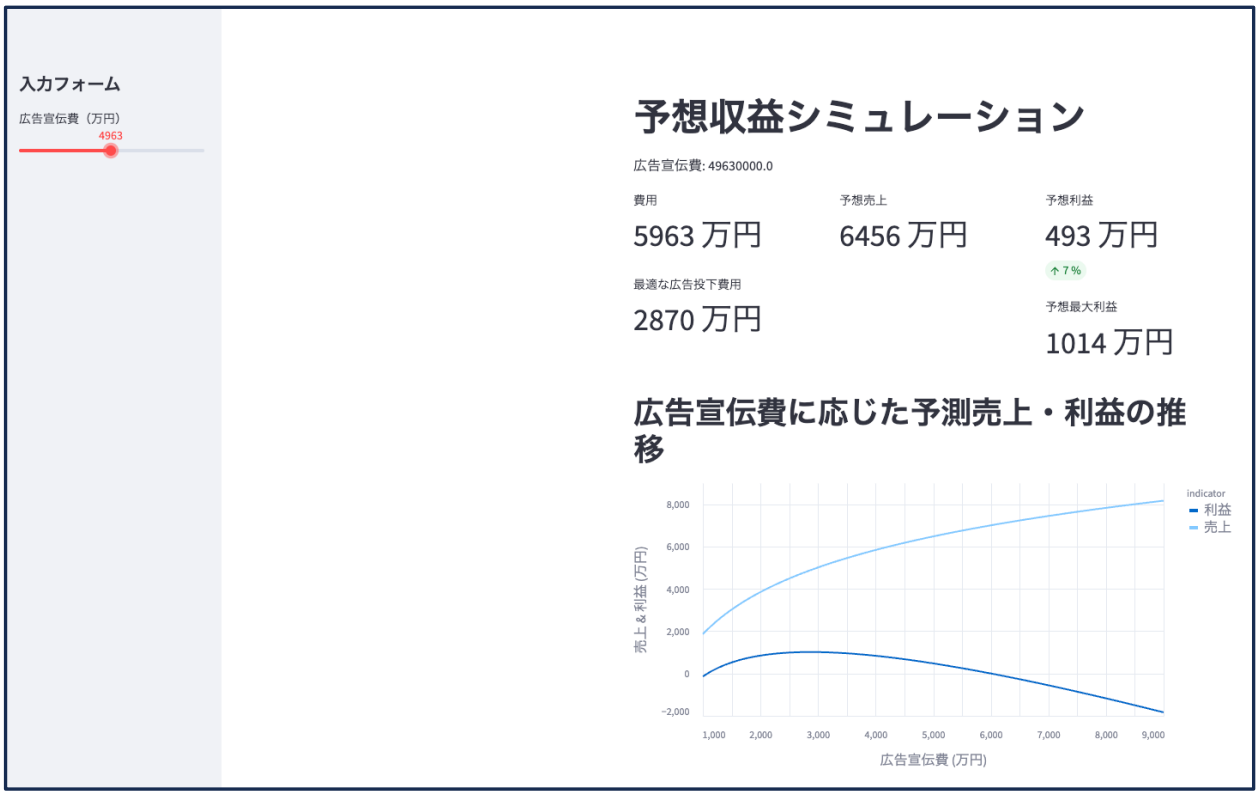
業務効率化④「動くプレゼン資料の作成!」

まず初めに Web アプリケーションを開発するためには何が必要だろう?かを考えてみよう!

Python には Streamlit というフレームワークがあり、データ分析 Web アプリに特化したフレームワークです。

要は、Streamlit を使用すればフロント・バックエンドともにできちゃうんですよ。

では、ここでは簡単に Web アプリを作成してみるんだけど、完成イメージはこうだ!



開発の流れは・・・
① 表示するのに必要なデータを JupyterLab で準備
② Python ファイルで Web アプリの見た目を実装

まずは、①の表示のために必要なデータを作っていこう！

まずは問題点から確認していこう！
今回は、広告宣伝費をいくら使用したら、売上がどんだけになるのか、そして利益がどれくらいになるのかをシミュレーションで作成していくよ！

問題を整理しよう！
① 広告宣伝費
　Web 広告を中心とした変動費（1000 万～9000 万でやっていく）
② その他の固定費
　広告宣伝費以外にかかる固定費（1000 万で固定）
③ 費用
　① + ②
④ 予想売上
　導き出した計算式を使用して、広告宣伝費を元に算出
⑤ 予想利益
　④ - ③

広告宣伝費をかければかけるほど予想売上は増加していくけど、予想利益はある一定のラインからは下がっていく。
今の時代、広告宣伝費をどれだけ抑えて売上に繋げるかが重要で、マーケティングやブランディングが鍵となっている。

また、クライアントからのヒアリングで売上重視か利益重視か、または広告宣伝費を抑えつつ目標の売上・利益を考えているのかは様々だよね。

ただ、今回は売上の最大化を目指して作成してみる！

広告宣伝費に応じた予想売上・予想利益を算出!

まずは、予想売上を算出する計算式だけど・・・・
y = 28700000×ln(x) - 444000000
x:広告宣伝費、y:予想売上　ln(x):x の自然対数

※要はこの 2 つの数字は「データから回帰で求めた係数」
適当に決めたのではなく、過去の広告費 xと 売上 yのペア（観測データ）を使って、
y≈β1ln(x)+β0
の $\beta_1 = 28{,}700{,}000$（傾き）と $\beta_0 = -444{,}000{,}000$（切片）を、最小二乗法で“いちばんズレが小さくなるように”自動で計算した結果なんだよね!

ざっくり直感

- 28,700,000（傾き）
  $\ln(x)$をちょっと増やしたときの、売上 yの増え幅。
  実務の感覚に直すと「広告費を 1%増やすと売上がだいたい 287,000 円ふえる（小さな変化の近似）」。
- -444,000,000（切片）
  式をそのまま延長したときの“$\ln(x) = 0$（$=x = 1$）付近での基準値”。
  基礎売上を別で持っていないため、低い広告費域だとマイナス推定になりがち。
  つまり「このモデルは“ある程度以上の広告費”の範囲で当てるための近似」というサイン。

なんでこんなデカい数字になるの？単位スケールの影響だから!
xを 円で入れれば $\ln(x)$は $\ln(10^6)$みたいに大きくなり、β_0 も大きく（負に）振れるから、もし xを 万円に変えると、同じデータでも係数値は変わるよ（モデルの意味は同じ）。
変換の関係は：

$$x' = \frac{x}{S}\ (\text{例：}S = 10{,}000\ \text{円/万円})\ \Rightarrow\ \ln(x) = \ln(x') + \ln(S)$$

$$y = \beta_1 \ln(x') + \underbrace{\left(\beta_0 + \beta_1 \ln S\right)}_{\text{新しい切片}}$$

つまり単位を変えると切片が $\beta_1 \ln S$だけズレるので、巨大な切片は珍しくないんだけど・・・と、わけわかんないと思うから y = 28700000×ln(x) - 444000000 は計算式でこれを使えばいいんだなで今はいい!

計算式は今のままではプログラム上エラーが出るからちょっと調整するよ！
y = 2.87E + 07 × ln(x) - 4.44E + 08

※E=「10 のべき乗」、2.0E+2 は 2.0×10^{2}=200.0

では、費用・予想売上・予想利益を準備していこう！

まずは、広告宣伝費を定義するため、変数仮置きで 2,000 万として進めよう。

```
pre_cost = 2.0E+7
pre_cost
```

実行結果

```
20000000.0
```

Web アプリを開発する際このままでは、扱いにくいので表現調整をする。

```
pre_cost = 2000*1.0E+4
pre_cost
```

実行結果

```
20000000.0
```

2000*1.0E+4 というのは、２千（2000）と１千（1.0E+4）という二つのパートに分けることで、2000 の部分を 1000 に帰れば１千万、100 とかえれば 100 万と柔軟に数値を変えれるよ！

次に、「その他の固定費」を変数 fix_cost として、1,000 万円を定義しよう。

```
fix_cost = 1000*1.0E+4
fix_cost
```

実行結果

```
10000000.0
```

全体の費用 cost は「広告宣伝費+その他の固定費」なので、

```
cost = pre_cost + fix_cost
cost
```

実行結果

```
30000000.0
```

予想売上を算出する関数を作成しよう！

y = 2.87E + 07 × ln(x) - 4.44E + 08

⇩

予想売上　= 2.8E + 07 × ln(広告宣伝費) - 4.44E + 08

自然対数

この予想売上の計算結果を返す処理を、calc_earnings()という関数に記述するために、math モジュールをインポートしてこの中に入っている math.log()関数を使用して、自然対数の値を簡単に計算するよ！

予想売上を算出する関数は・・・

```
def calc_earnings(pre_cost):
    earnings = 2.87E+07 * math.log(pre_cost) - 4.44E+08
    return int(earnings)
```

```
earnings = calc_earnings(pre_cost)
earnings
```

実行結果

```
38482669
```

計算結果が出力されたね！

計算結果が代入されている変数 earnings が整数になるように、int()で int 型に変換しているよね。

この結果は、広告宣伝費 ad_cost に 2,000 万を投資すると、予想売上は約 3,848 万になるよってことだ！

利益を算出する関数を作成する!
予想利益を算出する関数を calc_profit()として作成!

算出式は、予想利益=予想売上-費用だったよね!

```
def calc_profit(earnings, cost):
    profit = earnings - cost
    return int(profit)
```

```
profit = calc_profit(earnings, cost)
profit
```

実行結果

```
8482669
```

これは、予想売上が約 848 万円ってことを意味する。
これで、予想売上と予想利益を算出する関数ができたよね!併せて、予想利益率も計算しておこう!
利益率は、利益÷売上×100 で算出できるよ。

利益率の計算式は・・・
利益率　=　(利益/売上) ×100
であり、コードで書くとこうなる!

```
profit_ratio = int(profit / cost * 100)
profit_ratio
```

実行結果

```
28
```

グラフに表示するデータを準備しよう！

まず、今回の広告宣伝費は 1,000 万～9,000 万なので、この数字からグラフを表示するためには、折れ線グラフで点と点を繋いで曲線を作っていこう。
では、1,000 万～9,000 万を 1 万円単位の値をリストで用意したいよね。これは、range()関数と list()関数の組み合わせでできる！
range()は連番を作る関数で、range(開始番号、終了番号、[ステップ数])と記述する！
list()は、引数に指定した値をリスト型のデータに変換する。

まずは、1000～9000 の連番を作成して、先頭の 5 つを取得

```
list(range(1000, 9001, 1))[:5]
```

実行結果

```
[1000, 1001, 1002, 1003, 1004]
```

複数の広告宣伝費を持つリストを、変数 data_ad_cost に代入しておこう。

```
data_ad_cost = list(range(1000, 9001, 1))
```

次は、予想売上リストを準備

変数 data_ad_cost の値を元に、予想売上のリストを作成する。ここで使用する記述方法は内包表記という方法で作成するよ。
※内包表記とは、for 文を 1 行で記述して、リストを作成する方法

[リストに追加する値 for リストから取り出した要素 in 処理するリスト]

ここで、試しにリスト a の各要素を 10 倍した値を要素とするリスト b を作成してみよう！

```
a = [1,2,3,4,5]
b = [_a*10 for _a in a]
b
```

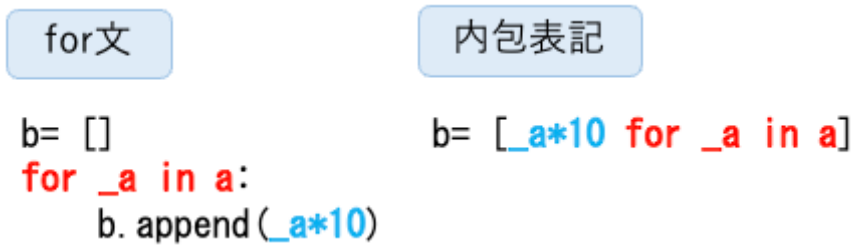
実行結果

```
[1000, 1001, 1002, 1003, 1004]
```

※ _の意味は、ただの変数名であり、「この値は使わない、一時的」といった意味も持っている。

通常の for 文で記述すると・・・

```
a = [1,2,3,4,5]
b = []
for _a in a:
    b.append(_a*10)
b
```



これを広告宣伝費のリスト data_ad_cost を使用して、予想売上のリスト data_earnings を作成しよう。

```
data_earnings = [calc_earnings(ad_cost*1.0E+04) for ad_cost in data_ad_cost]
data_earnings[:5]
```

実行結果

```
[18589345, 18618030, 18646687, 18675316, 18703916]
```

これで、約 1,858 万円、約 1,861 万円・・・・と、予想売上が入っているね!
内包表記を使用すると、コード量が短くなるんだけど、慣れていないから少し解説しよう!

① 広告宣伝費が入っているリスト data_ad_cost から一つの要素を取り出して、
② ad_cost に代入して、
③ 一つ一つの広告宣伝費 ad_cost に 1 万（1.0E+04）を掛けて、予想売上を算出する calc_earnings()関数に引数として渡す。
※1 万をかけるのは、ad_cost の値が 1,000 万～9,000 万だったよね。この数値を実際の大きさに戻すため 1 万を掛けて引数として渡しているんだよ。

予想利益のリストを準備しよう！
広告宣伝費 data_ad_cost し予想売上 data_earnings を使用して、予想利益 data_profit を準備するため zip()関数と for 文を組み合わせて取得しよう。

まずは、zip 関数と for 文の組み合わせを理解しよう！
zip()は、複数リストから一つずつ取得出る関数で、二つのリストを for 文で同時に操作したい時に使用するよ！
a,b というリストを準備してそれぞれのリストから一つずつ要素を取得してみる。
取得した値をそれぞれ_a,_b に代入して、掛け算した値をリスト c に追加するという流れでやってみるよ。

```
a = [1, 2, 3]
b = [4, 5, 6]
c = []
for _a, _b in zip(a, b):
    c.append(_a * _b)
c
```

実行結果

```
[4, 10, 18]
```

どういう事かというと・・・

- c[0] a と b の１つ目の要素である１と４を掛けている
- c[1] a と b の２つ目の要素である２と５を掛けている
- c[2] a と b の３つ目の要素である３と６を掛けている

では、費用と売上のリストを元に利益を計算してみよう！

利益は、費用と売上がないと計算できないよね。なので data_ad_cost と data_earnings からそれぞれ値を取得しないとダメだから、ここで zip()を使用する。
内包表記を使用して、予想利益のリストを作成しよう。

```
data_profit = [calc_profit(earnings, ad_cost*1.0E+04 + fix_cost) for earnings, ad_cost in zip(data_earnings, data_ad_cost)]
data_profit[:5]
```

実行結果

```
[-1410655, -1391970, -1373313, -1354684, -1336084]
```

data_earnings と data_ad_cost から一つずつ要素を取得し、earnings,ad_cost に代入している。この二つの値を利益算出のための関数 calc_profit の引数に指定しているけど、ad_cost は広告宣伝費であり、全体の費用問いことではない。
なので、ad_cost を 1 万倍（1.0E+04）した値に、その他固定費 fix_cost を足した結果を引数に指定しているよ！
この実行結果はマイナス値を出力しているけど、広告宣伝費が 1,000 万円では当初は赤字になるよっていうこと！
試しに 1000 番目の利益を確認してみよう！

```
data_profit[1000]
```

実行結果

```
8482669
```

確認のためのデータをプロットしよう！
広告宣伝費、予想売上、予想利益とリスト準備ができたけど、リストに入っているとイメージしにくいから、データをグラフ化してみよう！

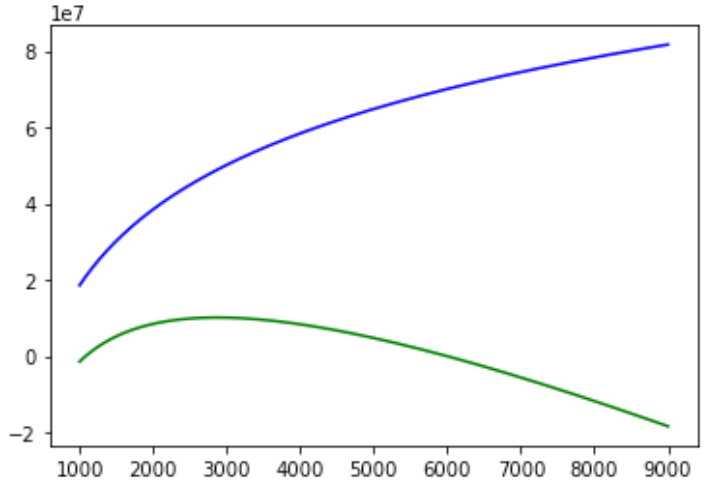
ここで使用するのが、matplotlib ライブラリだよ!インストールしよう！

```
!pip3 install matplotlib
```

では、このライブラリを使用して予想売上と利益をグラフで可視化してみよう！

```
import matplotlib.pyplot as plt
plt.plot(data_ad_cost, data_earnings, color='blue')
plt.plot(data_ad_cost, data_profit, color='green')
plt.show()
```

実行結果



今回は、確認のためだけだから見栄えは後ほど修正しよう！
グラフの横軸は広告宣伝費（data_ad_cost）、縦軸は予想売上と予想利益を表しているよ！

予想売上を示している青棒は、広告宣伝費が増加するにつれ増加する傾向にあり、それに対して予想利益を示している緑棒は、広告宣伝費が 2,000 万円を少し超えたところまでは増加しているけど、それ以降は下降していることがわかるね。
要は、広告宣伝費が 2,000 万円を少し超えた部分で、最大利益に到達しているよってこと！

予想最大利益と最適な広告宣伝費を算出する！

```
max_profit = max(data_profit)
max_profit
```

実行結果

```
10148100
```

利益の最大値 10,148,100 円が抽出できたね。
では、この最大利益を出すために必要な広告宣伝費はいくらになるのかを出力させるため、予想最大利益の値が data_profit の何番目の要素に入っているかを確認しよう。
その要素番号を取得して、広告宣伝費のリストインデックス番号として指定すると、最適な広告宣伝費が確認できる。

特定の値が入っているインデックス番号を取得するには、index()メソッドを使用すると確認できる。

```
data_profit.index(max_profit)
```

実行結果

```
1870
```

1870 という要素番号が取得できたね。これが最適な広告宣伝費だよってこと！

では、最適な広告宣伝費を取得しよう！

```
best_ad_cost = data_ad_cost[data_profit.index(max_profit)]
best_ad_cost
```

実行結果

```
2870
```

これで、2,870 万円が最適な広告宣伝費ということがわかったね！

これで必要なデータはすべて準備ができた！

あとは、見た目の調整なので一度コードをまとめておこう！

```
import math

ad_cost = 2000*1.0E+4
fix_cost = 1000*1.0E+4
cost = ad_cost + fix_cost

def calc_earnings(ad_cost):
    earnings = 2.87E+07 * math.log(ad_cost) - 4.44E+08
    return int(earnings)

def calc_profit(earnings, cost):
    profit = earnings - cost
    return int(profit)

earnings = calc_earnings(ad_cost)
profit = calc_profit(earnings, cost)
profit_ratio = int(profit / earnings * 100)

data_ad_cost = list(range(1000, 9001, 1))
data_earnings = [calc_earnings(ad_cost*1.0E+04) for ad_cost in data_ad_cost]
data_profit = [calc_profit(earnings, ad_cost*1.0E+04 + fix_cost) for
earnings, ad_cost in zip(data_earnings, data_ad_cost)]

max_profit = max(data_profit)
best_ad_cost = data_ad_cost[data_profit.index(max_profit)]
```

Streamlit の基本!

streamlit をインストールしよう!

```
pip install streamlit
```

インスーとるができたら次に、streamlit は st としてインポートする。
st.title(‘web アプリ開発)と記述してみよう。
st.title()はタイトルを作成したい時に使用するメソッドだよ!

```
import streamlit as st

st.title('webアプリ開発')
```

実行結果

表示されたかな?もし表示されなかったら少しタイムラグがあるので、リロードしたり再起動をしてみて!

では、次に通常のテキストを表示してみる。

```
st.write('こちらは通常のテキストだよ!')
```

実行結果

st.write()は、引数に指定した文字列を表示するメソッドで、テキストを表示する際に使用する。
また、文字列だけではなく Markdown も記述できるのが特徴だよ!
Markdown で記述してみよう。

```
st.write('''Markdown
- リスト1
- リスト2
- リスト3
''')
```

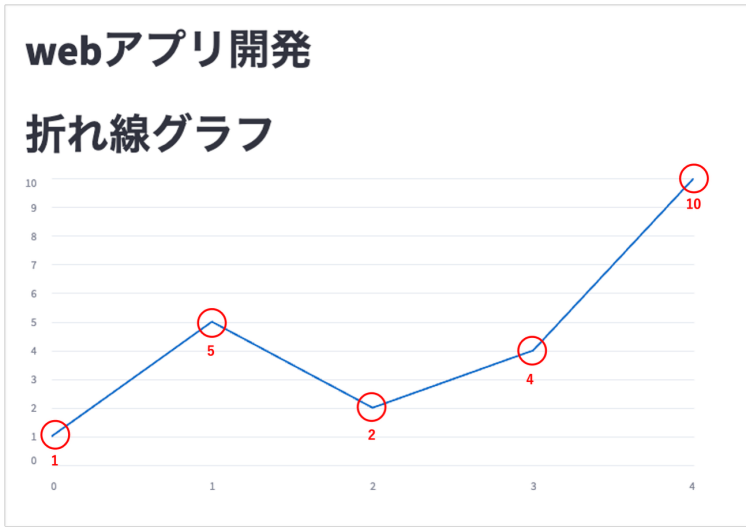
実行結果

折れ線グラフを出力してみよう！

テキストの表示方法をやってきたので次はグラフを表示したいので、基本形は st.〇〇という書き方で要素を出力してくれる。
まずは基本的な書き方として、折れ線グラフを出力してみよう！

```
st.write('# 折れ線グラフ')
a = [1,5,2,4,10]
st.line_chart(a)z
```

実行結果



streamlit で折れ線グラフを表示するには、st.line_chart()を使用する。
引数には、可視化したいデータを配列で入力するんだよ！

もっと詳しく勉強するにはドキュメントで勉強するといいよ！
Streamlit ドキュメント https://docs.streamlit.io

では、軽く streamlit を説明したので開発に戻ろう！

その前にまとめたコードを python ファイルに転記しよう！

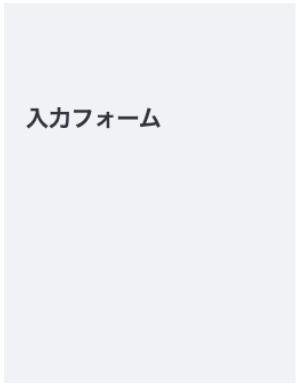
```
import streamlit as st
import math

ad_cost = 2000*1.0E+4
fix_cost = 1000*1.0E+4
cost = ad_cost + fix_cost

def calc_earnings(ad_cost):
    earnings = 2.87E+07 * math.log(ad_cost) - 4.44E+08
    return int(earnings)

def calc_profit(earnings, cost):
    profit = earnings - cost
    return int(profit)

earnings = calc_earnings(ad_cost)
profit = calc_profit(earnings, cost)
profit_ratio = int(profit / earnings * 100)

data_ad_cost = list(range(1000, 9001, 1))
data_earnings = [calc_earnings(ad_cost*1.0E+04) for ad_cost in data_ad_cost]
data_profit = [calc_profit(earnings, ad_cost*1.0E+04 + fix_cost) for
earnings, ad_cost in zip(data_earnings, data_ad_cost)]

max_profit = max(data_profit)
best_ad_cost = data_ad_cost[data_profit.index(max_profit)]
```

サイドバーにあるスライダーを作成!

では続いてサイドバーに表示されるスライダーを作成していこう。
サイドバーに何らかの要素を配置したい場合は、st.sidebar を使用するよ。
記述要領は、st.sidebar.〇〇と書く!
「入力フォーム」というテキスト見出しをコードの末端から記述して作成

```
st.sidebar.write('## 入力フォーム')
```

実行結果



サイドバーに「入力フォーム」というテキストが作成され折り畳みできるようになっているね!
では、スライダーを表示するため、同様に追記していくよ。
記述要領は、st.sidebar.slider(ラベル名、最小値、最大値)と書くよ。
広告宣伝費は、1,000 万〜9,000 万だったよね。

```
st.sidebar.slider('広告費（万円）', 1000, 9000)
```

実行結果

表示がされたら、この選択値を変数 ad_cost に代入しておこう!

```
ad_cost = st.sidebar.slider('広告宣伝費（万円）', 1000, 9000)
```

スライダーで取得している値は 1,000～9,000 なので、実際の値とは違うため、１万（1.0E+04）を掛けて、元の値に戻し広告宣伝費を ad_cost に代入しておこう。

```
st.sidebar.write('## 入力フォーム')
ad_cost = st.sidebar.slider('広告宣伝費（万円）', 1000, 9000)*1.0E+04
fix_cost = 1000*1.0E+4
cost = ad_cost + fix_cost

st.write(f'広告宣伝費: {ad_cost}')
```

実行結果

※ここで注意、コードの末端に記述しないで、元の ad_cost が定義されていた位置に置換する必要があるよ！

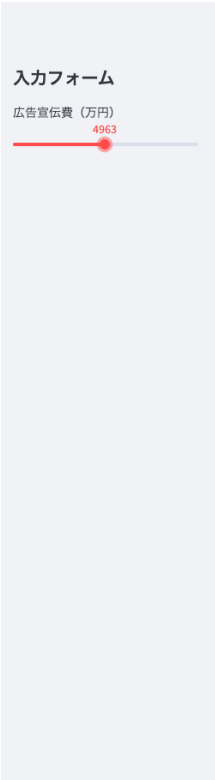
では、タイトルもつけておこう！

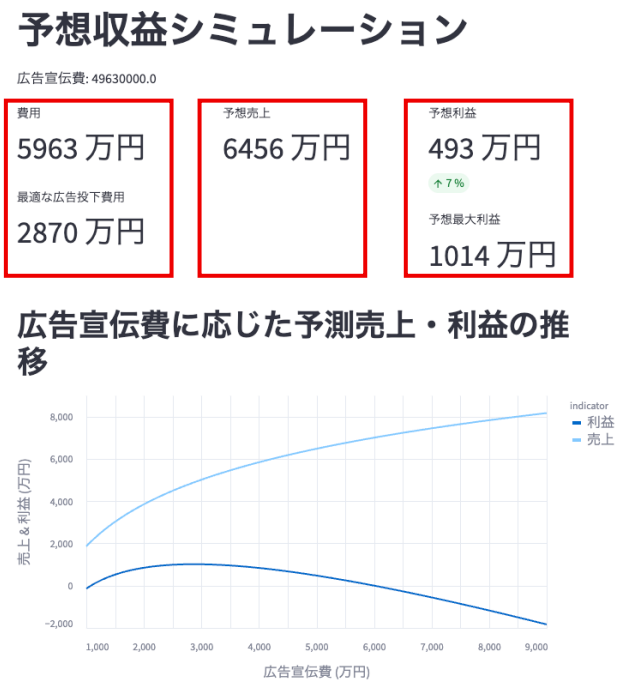
```
st.title('予想収益シミュレーション')
```

実行結果

画面中央に５つの指標を表示させる。

縦３列のレイアウトを作成する。
イメージはこう！

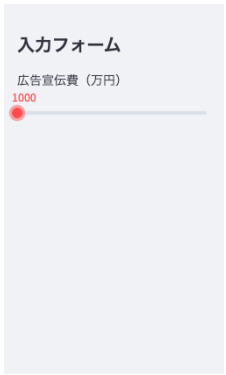




このように３つに分かれている状態を３カラム構成という。
この３カラムも作成してみよう！

```
col1, col2, col3 = st.columns(3)
col1.metric('費用', f'{int(cost/1.0E+04)} 万円')
col1.metric('最適な広告投下費用', f'{best_ad_cost} 万円')
col2.metric('予想売上', f'{int(earnings/1.0E+04)} 万円')
col3.metric('予想利益', f'{int(profit/1.0E+04)} 万円', f'{profit_ratio} %')
col3.metric('予想最大利益', f'{int(max_profit/1.0E+04)} 万円')
```

実行結果



予想収益シミュレーション

広告宣伝費: 10000000.0

費用
2000 万円

予想売上
1858 万円

予想利益
-141 万円
↓ -7 %

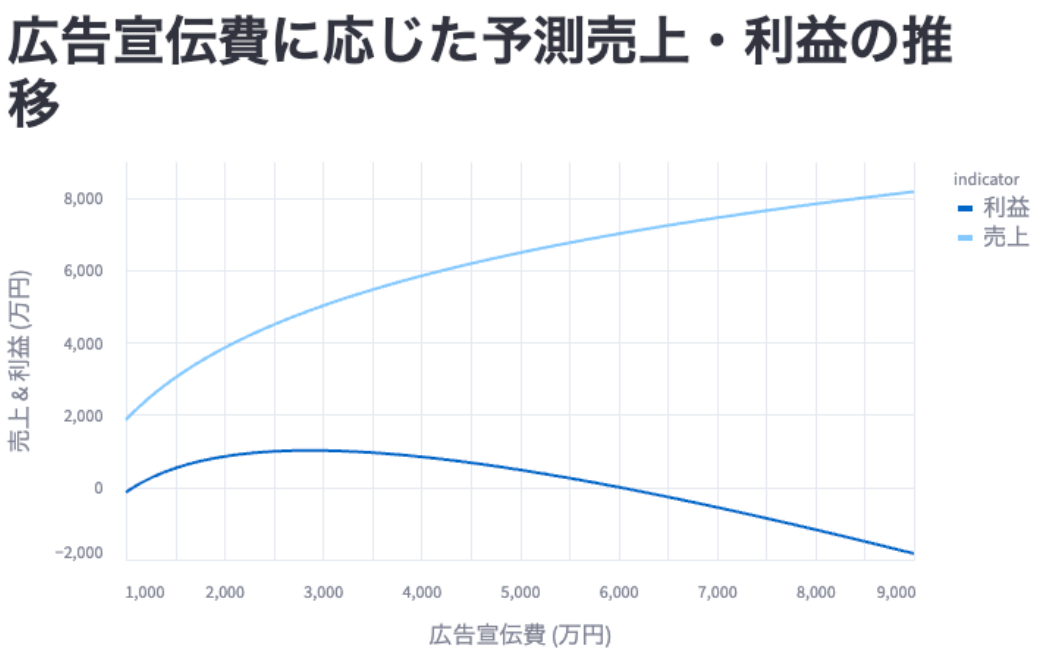
最適な広告投下費用
2870 万円

予想最大利益
1014 万円

コードを確認していく。
1 行目: col1, col2, col3 = st.columns(3)これは左からカラムを作成するよってこと。
2 行目以降:metric()を使用することで、指標を表示することができるようになり、引数には第１引数は指標名、第２引数には指標の値を指定するんだよ。
また、cost は元の桁数が表示されるため、cost/1.0E+04 と小さくして、int()に変換している。
そして、利益率 profit_ratio を入れることで画面に「↓-7%」と表示することができる。

推移グラフでデータの変化を可視化する！

数値を変えることでグラフも変動するようにしよう。



最初に紹介した st.line_chart()ではなくて、st.altair_chart()メソッドを使用する。
この Altair は、Streamlit 内に組み込まれているデータ可視化ライブラリメソッドで、st.line_chart()でも作成できるんだけど、カスタマイズ性が乏しいためこちらを使用する。
インストールをしてインポートしよう！

```
pip install altair
```

```
import pandas as pd
import altair as alt
```

Altair で可視化

この Altair は高度なグラフを可視化できるんだけど、読み取るデータ構造に制約があって、最初に出力されていたデータ構造にしないと読み込んでくれない!なので加工する必要があるんだよね。

	ad_cost	value	indicator
0	1000	18589345	売上
1	1001	18618030	売上
2	1002	18646687	売上
3	1003	18675316	売上
4	1004	18703916	売上
...	...	...	...
7996	8996	81636932	売上
7997	8997	81640122	売上
7998	8998	81643312	売上
7999	8999	81646501	売上
8000	9000	81649690	売上
	X 軸	Y 軸	ラベル

横軸=X 軸を表す広告宣伝費のカラム ad_cost、縦軸=Y 軸の予想売上・予想利益の値を表す value、各 value が予想売上・予想利益のどちらを表しているかを示す indicator のカラムを用意する必要がある。
要は、予想売上と予想利益をバラバラに用意するんじゃなくて、値をまとめて value に格納して、それぞれ何の値かをラベル付けする必要があるってこと。

この pandas のデータフレームを用意するためには、予想売上のデータフレーム df_earnings と、予想利益のデータフレーム df_profit をそれぞれに作成してから、この２つを結合するとい方法でやっていくよ!

予想売上のデータフレーム df_earnings を作成する！

```
df_earnings = pd.DataFrame()
df_earnings['ad_cost'] = data_ad_cost
df_earnings['value'] = data_earnings
df_earnings['indicator'] = '売上'
```

実行結果

	ad_cost	value	indicator
0	1000	18589345	売上
1	1001	18618030	売上
2	1002	18646687	売上
3	1003	18675316	売上
4	1004	18703916	売上
...	...	...	...
7996	8996	81636932	売上
7997	8997	81640122	売上
7998	8998	81643312	売上
7999	8999	81646501	売上
8000	9000	81649690	売上

では、コードを見ていこう！
1 行目： df_earnings = pd.DataFrame()は、df_earnings という空のデータフレームを作成して、３つのデータを挿入している。
① ad_cost カラムに「広告宣伝費」のリスト data_ad_cost
② value カラムに「予想売上」のリスト data_earnings
③ indicator カラムに「売上」という文字列

value の値が１万倍になっているから後にちゃんと処理しよう！

予想利益のデータフレーム df_profit を作成する！

同じ要領で作成しよう。

```
df_profit = pd.DataFrame()
df_profit['ad_cost'] = data_ad_cost
df_profit['value'] = data_profit
df_profit['indicator'] = '利益'
```

実行結果

	ad_cost	value	indicator
0	1000	-1410655	利益
1	1001	-1391970	利益
2	1002	-1373313	利益
3	1003	-1354684	利益
4	1004	-1336084	利益
...	...	...	...
7996	8996	-18323068	利益
7997	8997	-18329878	利益
7998	8998	-18336688	利益
7999	8999	-18343499	利益
8000	9000	-18350310	利益

大丈夫そうだね！

データフレームを結合!
pandas のデータフレームを結合するためには、縦に結合したい場合 pd.concat()を使用しするよ。
※横結合の場合: df = pd.concat([df1, df2], axis=1)

```
df = pd.concat([df_earnings, df_profit])
df['value'] = df['value'] / 1.0E+04v
```

pd.concat()の引数には、結合したいデータフレームを上から指定すれば OK!
2 行目の 1 万で割っているのは、理解しやすい数値に変換するためだよ!

Altair によるグラフ設定!
データが揃ったのでグラフ化していこう。

```
chart = alt.Chart(df).mark_line().encode(
    alt.X('ad_cost', title='広告宣伝費 (万円)'),
    alt.Y('value', title='売上 & 利益 (万円)'),
    color='indicator'
).configure_axis(
    labelFontSize=12,
    titleFontSize=16,
).configure_legend(
    titleFontSize=12,
    labelFontSize=16,
)
```

うん、とりあえずコードを書いてみて、そこから説明していく。

```
chart = alt.Chart(df).mark_line().encode(
    alt.X('ad_cost', title='広告宣伝費 (万円)'),
    alt.Y('value', title='売上 & 利益 (万円)'),
    color='indicator'
)
```

さっきインポートしたimport altair as altは、altライブラリの中にChart()というグラフを作成するクラスがある。この引数に可視化したいデータdfを指定することでセット完了となる。
そして、折れ線グラフとして.mark_line()を繋げて記述し.encode()X軸やY軸に使用する列等の指定を行う関数だよ。

① alt.X('ad_cost', title='広告宣伝費 (万円)'),は、横軸=X 軸に広告宣伝費 ad_cost をタイトルに「広告宣伝費 (万円)」を設定
② alt.Y('value', title='売上 & 利益 (万円)'),縦軸=Y 軸に予想売上・予想利益の値が入っている value を指定し、縦軸のタイトルを「売上 & 利益 (万円)」と設定
③ color='indicator'は、中に入っている値を元に色分けしたグラフを表示

```
.configure_axis(
    labelFontSize=12,
    titleFontSize=16,
)
```

次にこの部分は、先頭にピリオドがあるので、さっきの.encode()に繋げて記述していることとなる。.configure_axis()は、名の通り axis=軸を configure=設定するというメソッドで、ラベルのフォントを設定する labelFontSize と軸タイトルのフォントを設定する titleFontSize ということ。

```
.configure_legend(
    titleFontSize=12,
    labelFontSize=16,
)
```

そして.configure_legend()は、legend=凡例の configure=設定をするメソッドだよ。

表示させよう！
コードの末端にグラフを表示するコードを追加しよう！

```
st.write('## 広告宣伝費に応じた予測売上・利益の推移')
st.altair_chart(chart, use_container_width=True)
```

グラフを表示する st.altair_chart()の第１引数に Altair で作成したグラフ chart を指定し、第２引数に use_container_width=True を指定すねことでグラフの横幅を列の幅に合わせる。

完成コード

```
import streamlit as st
import math
import pandas as pd
import altair as alt

st.title('予想収益シミュレーション')
st.sidebar.write('## 入力フォーム')
ad_cost = st.sidebar.slider('広告宣伝費（万円）', 1000, 9000)*1.0E+04
fix_cost = 1000*1.0E+4
cost = ad_cost + fix_cost

st.write(f'広告宣伝費: {ad_cost}')

def calc_earnings(ad_cost):
    earnings = 2.87E+07 * math.log(ad_cost) - 4.44E+08
    return int(earnings)

def calc_profit(earnings, cost):
    profit = earnings - cost
    return int(profit)

earnings = calc_earnings(ad_cost)
profit = calc_profit(earnings, cost)
profit_ratio = int(profit / earnings * 100)

data_ad_cost = list(range(1000, 9001, 1))
data_earnings = [calc_earnings(ad_cost*1.0E+04) for ad_cost in data_ad_cost]
data_profit = [calc_profit(earnings, ad_cost*1.0E+04 + fix_cost) for earnings, ad_cost in
zip(data_earnings, data_ad_cost)]

max_profit = max(data_profit)
best_ad_cost = data_ad_cost[data_profit.index(max_profit)]

col1, col2, col3 = st.columns(3)
col1.metric('費用', f'{int(cost/1.0E+04)} 万円')
col1.metric('最適な広告投下費用', f'{best_ad_cost} 万円')
col2.metric('予想売上', f'{int(earnings/1.0E+04)} 万円')
col3.metric('予想利益', f'{int(profit/1.0E+04)} 万円', f'{profit_ratio} %')
col3.metric('予想最大利益', f'{int(max_profit/1.0E+04)} 万円')

df_earnings = pd.DataFrame()
df_earnings['ad_cost'] = data_ad_cost
df_earnings['value'] = data_earnings
df_earnings['indicator'] = '売上'
```

```
df_profit = pd.DataFrame()
df_profit['ad_cost'] = data_ad_cost
df_profit['value'] = data_profit
df_profit['indicator'] = '利益'

df = pd.concat([df_earnings, df_profit])
df['value'] = df['value'] / 1.0E+04

chart = alt.Chart(df).mark_line().encode(
    alt.X('ad_cost', title='広告宣伝費 (万円)'),
    alt.Y('value', title='売上 & 利益 (万円)'),
    color='indicator'
).configure_axis(
    labelFontSize=12,
    titleFontSize=16,
).configure_legend(
    titleFontSize=12,
    labelFontSize=16,
)

st.write('## 広告宣伝費に応じた予測売上・利益の推移')
st.altair_chart(chart, use_container_width=True)
```

最後に、グラフをスライドバーと連動させるためには、これを追加したら動くんだけど、どっかのコード１箇所変える必要があるからやってみよう！

```
xmax = int(ad_cost / 1.0E+04)  # 追加
df_view = df[df["ad_cost"] <= xmax]  # 追加
```